Run Logging & Workflow Tracker

Implementation guide for report developers — LRN Report Engine

Version 3.1 | Updated 05 August 2026 | Database: `LRNMaster`

1. What this document is for

Every process that produces a report — C#, Python, SSIS, or a manual script — must record two things: **what it did while running** and **how it finished**. Those go to two tables in `LRNMaster`, and are always written through stored procedures:

Table	Written through	Holds
`dbo.ReportRunIdInfoLog`	`dbo.usp_ReportRunIdInfoLog_Insert`	The progress trail: started, milestones, warnings, errors, ended. Many rows per report.
`dbo.ReportsWorkflowTracker`	`dbo.usp_ReportsWorkflowTracker_Upsert`	The outcome: one row per RunId + Lab + Report, showing Success / Failed / Skipped / InProgress. This is what the dashboard reads.

> **Never INSERT or UPDATE these two tables directly.** The procedures resolve the lab, the week folder and the report type for you, validate what you send, and keep the write idempotent. Direct writes bypass all of that and produce rows the dashboard cannot interpret.

2. The four steps

Step	Do this	Using
1	Get the RunId to work under	`dbo.sp_GetRecentSuccessRunByLab`
2	Stop if this report is already done for that RunId	`SELECT on dbo.ReportsWorkflowTracker`
3	Log progress as you go, and mark the tracker `InProgress`	`dbo.usp_ReportRunIdInfoLog_Insert`
4	Record the outcome — Success, Failed or Skipped	`dbo.usp_ReportsWorkflowTracker_Upsert`

Step 3 repeats as often as you like during the run. Step 4 runs once at the start (`InProgress`) and once at the end (the final status).

3. Step 1 — Get the RunId

A RunId identifies one lab's processing run, e.g. `R20260804ELX0072` . Your report attaches itself to the lab's most recent successful run rather than inventing an id.

```
EXEC dbo.sp_GetRecentSuccessRunByLab @LabName = 'Elixir';
```

Returns `RunID`, `LabName`, `OverallStatus`, `StartTimeIST`, `EndTimeIST` for the latest run with `OverallStatus = 'SUCCESS'`.

Parameter	Type	Notes
`@LabName`	NVARCHAR(50), optional	Matched with `LIKE '%name%'`, so a partial name works. Leave it NULL to get the latest successful run for *every* lab, one row each.

Handle the empty result. A lab with no successful run yet returns no rows. That is not an error — there is simply nothing to report on. Exit quietly rather than continuing with an empty RunId, which both procedures reject anyway.

4. Step 2 — Do not repeat work already done

Reports are re-triggered routinely. Before doing any work, check whether this report already succeeded for this RunId:

```
SELECT COUNT(1)
FROM   dbo.ReportsWorkflowTracker
WHERE  RunId      = 'R20260804ELX0072'
  AND  ReportName = 'Payer Policy Validation'
  AND  Status     = 'Success';
```

A count of **1** means it is done — stop. A count of **0** means it has never run, or it previously failed or was skipped — carry on.

Include `Status = 'Success'`. Without it a previous `Failed` or `InProgress` row also returns 1, and your report would refuse to retry the very run that needs retrying. Filter on `ReportName` (the value you pass in step 4); `ReportType` holds the same text, resolved from the master.

5. Step 3 — Log progress

```
EXEC dbo.usp_ReportRunIdInfoLog_Insert
     @RunId          = 'R20260804INH0041',
     @ReportType     = 'Forecasting',
     @SourceSystem   = 'InHealth',
     @SourceFileName = 'InHealth_Master_07.29.2026.xlsx',
     @LogType        = 'Info',
     @LogMessage     = 'Forecast completed. Rows=195161.',
     @CreatedBy      = 'Forecast Python Processor';
```

Parameter	Type	Notes
`@RunId`	VARCHAR(30), required	From step 1. Blank is rejected.

`@ReportType`	VARCHAR(100), required	Free text here, unlike step 4. Use the name from section 7 so the log joins to the master; a finer-grained label is allowed when it genuinely helps.
`@SourceSystem`	VARCHAR(100), required	The lab or upstream system, e.g. `Elixir`.
`@SourceFileName`	NVARCHAR(400), optional	File being processed, when there is one.
`@LogType`	VARCHAR(50), required	Exactly one of `Start`, `Info`, `Warning`, `Error`, `End`. Anything else is rejected.
`@LogMessage`	NVARCHAR(MAX), optional	What happened. Include counts, table names, durations.
`@CreatedBy`	VARCHAR(100), required	The process name, not a person, e.g. `Forecast Python Processor`.

When to write each LogType

LogType	Write it when
`Start`	The report begins. One per report per run.
`Info`	A meaningful milestone: source read, N rows produced, table loaded.
`Warning`	Something is off but the run continues — missing optional input, unmapped columns, a partial refresh.
`Error`	The failure. Put the **full exception text** here; the tracker only carries a short remark.
`End`	The report finishes, whether it succeeded or failed. One per report per run.

> **Never log patient data.** Log messages are read widely and retained. Counts, file names, table names and durations are fine. Patient names, dates of birth, addresses, member or policy numbers and Social Security Numbers must never appear in `@LogMessage` — naming a *column* is fine, printing its *value* is not.

6. Step 4 — Record the outcome

```
EXEC dbo.usp_ReportsWorkflowTracker_Upsert
     @RunId       = 'R20260804INH0041',
     @ReportName  = 'Forecasting',
     @Status      = 'Success',
     @RowCount    = 195161,
     @StartedOn   = '2026-08-05 02:07:53.033',
     @CompletedOn = '2026-08-05 02:41:10.220',
     @Remarks     = NULL,
     @CreatedBy   = 'Forecast Python Processor';
```

Parameter	Type	Notes
`@RunId`	VARCHAR(30), required	From step 1.
`@ReportName`	VARCHAR(200), required	Must match `dbo.ReportTypeMaster` exactly — see section 7. An unknown name is rejected.

`@Status`	VARCHAR(50), required	Exactly one of `Success`, `Failed`, `Skipped`, `InProgress`. Case matters.
`@RowCount`	BIGINT, optional	Rows produced or loaded. Leave NULL when the report has no natural count.
`@StartedOn`	DATETIME2(3), optional	When your report started. IST.
`@CompletedOn`	DATETIME2(3), optional	When it ended. Leave NULL on the opening `InProgress` call.
`@Remarks`	NVARCHAR(MAX), optional	Short reason for a `Failed` or `Skipped` row. The full error belongs in the info log.
`@CreatedBy`	VARCHAR(100), required	The process name.
`@LabId`	INT, optional	Only needed when the RunId is not yet in `dbo.LRN_Run_Log`. Normally omit it — the procedure resolves the lab from the run log.
`@WeekFolder`	VARCHAR(200), optional	Same — resolved for you when omitted.

The call is idempotent. One row exists per RunId + Lab + Report, so calling it again for the same report updates that row instead of adding another. Call it twice by design: `InProgress` when you start, then the final status when you finish.

Choosing the status

Status	Use it when
`InProgress`	Work has begun. Written immediately after step 2 passes.
`Success`	The report produced its output. Send `@RowCount` where one exists.
`Failed`	It did not complete. Short reason in `@Remarks`, full exception in the info log.
`Skipped`	It deliberately did not run — nothing to process, switched off, no source file. Always give the reason in `@Remarks`.

A missing row is worse than a Skipped one. If your report decides it has nothing to do, write `Skipped` with a reason. An absent row is indistinguishable from a crashed process, and someone will spend time investigating it.

7. Report names

Use these **exactly** in `@ReportName`. The procedure resolves `ReportTypeId` from the name and rejects anything not listed. Ids are shown for reference — always send the name, never the id.

ReportTypeId	ReportTypeName	ReportTypeId	ReportTypeName
1	Claim Level Master	8	Line Level Master
2	Clinic Summary	9	LIS Summary
3	Coding Validation	10	Payer Policy Validation

4	Collection Summary	11	Prediction Analysis
5	Denial Report	12	Production Summary
6	Executive Summary	13	Sales Rep Summary
7	Forecasting	14	Error Log

The list is always current here:

```
SELECT ReportTypeId, ReportTypeName
FROM   dbo.ReportTypeMaster
WHERE  IsActive = 1
ORDER  BY DisplayOrder;
```

Need a name that is not on the list? Ask for it to be added to `dbo.ReportTypeMaster`. Do not invent one — the tracker will reject it.

8. Complete example

One report, start to finish, in the order your code should call them.

```
/* ---- Step 1: the RunId to work under ---- */
DECLARE @RunId VARCHAR(30), @Lab VARCHAR(100) = 'Elixir';

SELECT TOP (1) @RunId = RunID
FROM   dbo.LRN_Run_Log
WHERE  LabName LIKE '%' + @Lab + '%' AND OverallStatus = 'SUCCESS'
ORDER  BY EndTimeIST DESC;          -- or: EXEC dbo.sp_GetRecentSuccessRunByLab @LabName = @Lab;

IF @RunId IS NULL RETURN;           -- no successful run yet: nothing to report on

/* ---- Step 2: already done? ---- */
IF EXISTS (SELECT 1 FROM dbo.ReportsWorkflowTracker
           WHERE RunId = @RunId AND ReportName = 'Forecasting' AND Status = 'Success')
    RETURN;

/* ---- Step 3: opening entries ---- */
DECLARE @StartedOn DATETIME2(3) = SYSDATETIME();

EXEC dbo.usp_ReportRunIdInfoLog_Insert
     @RunId        = @RunId,
     @ReportType   = 'Forecasting',
     @SourceSystem = @Lab,
     @LogType      = 'Start',
     @LogMessage   = 'Forecast started.',
     @CreatedBy    = 'Forecast Python Processor';

EXEC dbo.usp_ReportsWorkflowTracker_Upsert
     @RunId      = @RunId,
     @ReportName = 'Forecasting',
```

```
     @Status     = 'InProgress',
     @StartedOn  = @StartedOn,
     @CreatedBy  = 'Forecast Python Processor';

/* ---- ... your report does its work here ... ---- */

/* ---- Step 4: the outcome ---- */
EXEC dbo.usp_ReportRunIdInfoLog_Insert
     @RunId        = @RunId,
     @ReportType   = 'Forecasting',
     @SourceSystem = @Lab,
     @LogType      = 'Info',
     @LogMessage   = 'Forecast completed. Rows=195161.',
     @CreatedBy    = 'Forecast Python Processor';

EXEC dbo.usp_ReportsWorkflowTracker_Upsert
     @RunId       = @RunId,
     @ReportName  = 'Forecasting',
     @Status      = 'Success',
     @RowCount    = 195161,
     @StartedOn   = @StartedOn,
     @CompletedOn = SYSDATETIME(),
     @CreatedBy   = 'Forecast Python Processor';

EXEC dbo.usp_ReportRunIdInfoLog_Insert
     @RunId        = @RunId,
     @ReportType   = 'Forecasting',
     @SourceSystem = @Lab,
     @LogType      = 'End',
     @LogMessage   = 'Forecast processing ended.',
     @CreatedBy    = 'Forecast Python Processor';
```

The failure path

```
/* In your catch / except block */
EXEC dbo.usp_ReportRunIdInfoLog_Insert
     @RunId        = @RunId,
     @ReportType   = 'Forecasting',
     @SourceSystem = @Lab,
     @LogType      = 'Error',
     @LogMessage   = @FullExceptionText,      -- everything: message, stack, inner exception
     @CreatedBy    = 'Forecast Python Processor';

EXEC dbo.usp_ReportsWorkflowTracker_Upsert
     @RunId       = @RunId,
     @ReportName  = 'Forecasting',
     @Status      = 'Failed',
     @StartedOn   = @StartedOn,
     @CompletedOn = SYSDATETIME(),
     @Remarks     = @ShortErrorMessage,       -- one line, not the stack
     @CreatedBy   = 'Forecast Python Processor';
```

> **Logging must never break your report.** Wrap these calls so a logging failure is caught and written to your own file log instead of aborting the run. A logging outage should cost you a log row, not a night's processing.

9. Rules and common mistakes

Mistake	What happens / do this instead
Missing comma between parameters, e.g. `@Remarks='x'` then `@CreatedBy=...` on the next line	Syntax error, nothing is logged. Check every line but the last ends with a comma.
`@Status = 'status'`, `'success'`, `'COMPLETED'`	Rejected. Exactly `Success`, `Failed`, `Skipped`, `InProgress`.
`@LogType = 'info'` in the wrong case, or a value such as `'Debug'`	Rejected. Exactly `Start`, `Info`, `Warning`, `Error`, `End`.
A report name spelled loosely, e.g. `'LIS summary'`	Rejected — the name must match `dbo.ReportTypeMaster` character for character.
`@CreatedBy` set to a person's name or left out	Required, and it is the *process* identity. Use the service or script name.
Writing only on success	Failures and no-ops need rows too, as `Failed` and `Skipped`.
The full stack trace in `@Remarks`	Keep `@Remarks` to one line; the full text goes in the info log with `@LogType='Error'`.
Inserting into the tables directly	Bypasses lab, week-folder and report-type resolution. Always call the procedures.
A RunId your process invented	Must come from `dbo.LRN_Run_Log`, or the tracker rejects it unless you also pass `@LabId`.

10. Errors you may see

Message	Meaning
`@RunId is required.`	Blank or whitespace RunId. Step 1 returned nothing and the code carried on.
`@LogType must be Start, Info, Warning, Error or End.`	Wrong `@LogType` value.
`@Status must be Success, Failed, Skipped or InProgress.`	Wrong `@Status` value.
`"X" is not an active report in dbo.ReportTypeMaster.`	Report name typo, or the name needs adding to the master.
`RunId "X" was not found in dbo.LRN_Run_Log and no @LabId was supplied.`	The RunId does not exist yet. Use one from step 1, or pass `@LabId`.

11. Checking your own work

```
/* The outcome row the dashboard reads */
SELECT RunId, LabName, ReportName, Status, [RowCount], StartedOn, CompletedOn, Remarks
FROM   dbo.ReportsWorkflowTracker
WHERE  RunId = 'R20260804ELX0072'
ORDER  BY ReportName;

/* The trail your report left */
SELECT LogType, LogMessage, CreatedOn
FROM   dbo.ReportRunIdInfoLog
WHERE  RunId = 'R20260804ELX0072' AND ReportType = 'Forecasting'
ORDER  BY ReportRunIdInfoLogId;
```

A correctly instrumented report shows a `Start`, at least one `Info`, an `End`, and exactly one tracker row that is no longer `InProgress`.

12. For .NET services

C# services in this solution should not hand-write the SQL. Two classes already wrap both procedures, swallow their own failures and fall back to the file log:

Class	Use
`ReportRunIdInfoLogger`	`StartAsync`, `InfoAsync`, `WarningAsync`, `ErrorAsync`, `EndAsync`. `ErrorAsync` takes the exception and writes the full text for you.
`ReportsWorkflowTrackerRepository`	`UpsertAsync(...)` for step 4. Status constants live in `WorkflowStatus`; report names in `WorkflowReportNames`.

Both are registered in DI — inject them rather than newing them up. Use the constants instead of string literals so a rename is a compile error rather than a runtime rejection.

Appendix — what changed in this revision

- Corrected the SQL examples: the missing comma before `@CreatedBy`, and `@Status='status'` replaced with a real value.
- Report id list corrected to match `dbo.ReportTypeMaster`, including `14 Error Log`, which was absent.
- Added the full parameter tables for both procedures, including `@SourceFileName`, `@LabId` and `@WeekFolder`, which were not documented.
- Added the permitted values for `@LogType` and `@Status`, and the errors raised when they are wrong.
- Step 2 now filters on `Status = 'Success'`, so a failed run can be retried.
- Added the failure path, the `Skipped` rule, the complete worked example, verification queries and the .NET helper classes.
- Added the rule against writing patient data to log messages.